Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №6
Варіант 9
На тему
“Замикання та декоратори як HOF”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- створювати замикання та розуміти, як вони утримують стан;
- писати параметризовані декоратори як фабрики декораторів;
- реалізувати декоратори таймінгу та валідації з ін'єкцією залежностей.

Теоретичні відомості:

- Замикання - функція, що пам'ятає змінні з навколишньої області видимості навіть після її завершення.
- Декоратор - функція вищого порядку decorator(func) -> wrapper; параметризований декоратор - фабрика, що повертає декоратор.
- functools.wraps зберігає __name__, __doc__ і __wrapped__ обгорнутої функції.
- ParamSpec і TypeVar зберігають типи аргументів і результату в сигнатурі обгортки.

Завдання (варіант 9 - Декоратор валідації аргументів):

Фабрика валідаторів-closures, параметризований @validate і @timed. Перевіряти діапазони та типові умови до виконання функції; валідатори передавати як Callable; тестувати помилки і коректні дані.

Хід роботи

У closures.py реалізовано замикання: make_counter зі змінним станом через nonlocal, make_logger, якому місце запису передається як залежність, make_memory_sink (пара замикань запису та читання історії) і фабрику валідаторів in_range, positive, non_empty, is_type, one_of, is_number. Кожен валідатор захоплює свої параметри і повертає текст помилки або None.

У decorators.py реалізовано три параметризовані декоратори. @validate(**rules) через inspect.signature зв'язує аргументи з параметрами, враховує значення за замовчуванням, перевіряє кожен аргумент усіма його валідаторами і збирає помилки в ValidationError. Правило для неіснуючого параметра дає TypeError ще під час декорування.

@timed(label, unit, logger, clock) вимірює час через ін'єктований годинник і пише лог у блоці finally, тобто навіть при винятку. @count_calls(logger, counter_factory) рахує виклики через замикання make_counter.

Усі обгортки використовують functools.wraps і типізовані через ParamSpec та TypeVar.

Тести працюють без реального часу (фейковий годинник і логер), перевіряють збереження __name__, __doc__, __wrapped__, стан замикань, граничні значення валідаторів і те, як порядок композиції декораторів впливає на підрахунок невалідних викликів.

Файл closures.py

closures.py рядки 1-57

```python
"""Замикання ЛР6, варіант 9: лічильник, логер і фабрика валідаторів."""

from collections.abc import Callable
from numbers import Real

Validator = Callable[[object], str | None]
"""Валідатор повертає текст помилки або None, якщо значення коректне."""


def make_counter(start: int = 0) -> Callable[[], int]:
    """Лічильник викликів: стан value зберігається в замиканні через nonlocal."""
    value = start

    def inc() -> int:
        """Збільшує лічильник і повертає нове значення."""
        nonlocal value
        value += 1
        return value

    return inc


def make_logger(prefix: str, sink: Callable[[str], None]) -> Callable[[str], None]:
    """Логер з префіксом; куди писати (sink) передається як залежність."""

    def log(message: str) -> None:
        """Пише повідомлення з префіксом."""
        sink(f"[{prefix}] {message}")

    return log


def make_memory_sink() -> tuple[Callable[[str], None], Callable[[], tuple[str, ...]]]:
    """Пара замикань: запис повідомлень і читання накопиченої історії."""
    messages: list[str] = []

    def write(message: str) -> None:
        """Зберігає повідомлення."""
        messages.append(message)

    def history() -> tuple[str, ...]:
        """Повертає незмінну копію історії."""
        return tuple(messages)

    return write, history


def is_number() -> Validator:
    """Значення має бути числом (але не bool)."""
    return lambda x: (
        None if isinstance(x, Real) and not isinstance(x, bool) else "має бути числом"
    )


def in_range(lo: float, hi: float) -> Validator:
    """Число в межах [lo, hi]; межі захоплені замиканням."""
```

closures.py рядки 58-102

```python
    def check(x: object) -> str | None:
        """Перевіряє діапазон."""
        if not isinstance(x, Real) or isinstance(x, bool):
            return "має бути числом"
        return None if lo <= float(x) <= hi else f"має бути в межах [{lo}, {hi}]"

    return check


def positive() -> Validator:
    """Число більше нуля."""

    def check(x: object) -> str | None:
        """Перевіряє, що число додатне."""
        if not isinstance(x, Real) or isinstance(x, bool):
            return "має бути числом"
        return None if float(x) > 0 else "має бути додатним"

    return check


def non_empty() -> Validator:
    """Рядок або колекція не порожні."""

    def check(x: object) -> str | None:
        """Перевіряє непорожність."""
        if isinstance(x, str):
            return None if x.strip() else "не може бути порожнім"
        if isinstance(x, (list, tuple, set, frozenset, dict)):
            return None if x else "не може бути порожнім"
        return "має бути рядком або колекцією"

    return check


def is_type(*types: type) -> Validator:
    """Значення має один із дозволених типів."""
    names = ", ".join(t.__name__ for t in types)
    return lambda x: None if isinstance(x, types) else f"має бути типу {names}"


def one_of(*options: object) -> Validator:
    """Значення входить до дозволеного набору."""
    allowed = tuple(options)
    return lambda x: None if x in allowed else f"має бути одним із {allowed}"
```

Файл decorators.py

decorators.py рядки 1-57

```python
"""Параметризовані декоратори ЛР6, варіант 9: @validate, @timed, @count_calls."""

import functools
import inspect
import time
from collections.abc import Callable, Sequence
from typing import Literal, ParamSpec, TypeVar

from closures import Validator, make_counter

P = ParamSpec("P")
R = TypeVar("R")


class ValidationError(ValueError):
    """Аргументи функції не пройшли валідацію."""


def validate(
    **rules: Validator | Sequence[Validator],
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Фабрика декоратора: перевіряє аргументи до виклику функції.

    Приклад: @validate(age=in_range(0, 120), name=[non_empty(), is_type(str)]).
    """
    checks = {
        name: tuple(rule) if isinstance(rule, Sequence) else (rule,)
        for name, rule in rules.items()
    }

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        """Звичайний декоратор із захопленими правилами."""
        signature = inspect.signature(func)
        unknown = set(checks) - set(signature.parameters)
        if unknown:
            raise TypeError(f"{func.__name__} не має параметрів {sorted(unknown)}")

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            """Перевіряє аргументи і викликає оригінальну функцію."""
            bound = signature.bind(*args, **kwargs)
            bound.apply_defaults()
            errors = [
                f"{name}: {message}"
                for name, validators in checks.items()
                for check in validators
                if (message := check(bound.arguments[name])) is not None
            ]
            if errors:
                raise ValidationError(f"{func.__name__}: " + "; ".join(errors))
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

decorators.py рядки 58-105

```python
def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None] = print,
    clock: Callable[[], float] = time.perf_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Фабрика декоратора: логує час виконання (навіть при винятку)."""
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        """Звичайний декоратор із захопленими налаштуваннями."""
        name = label or func.__name__

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            """Вимірює час виклику через ін'єктований clock."""
            start = clock()
            try:
                return func(*args, **kwargs)
            finally:
                logger(f"[timed] {name} took {(clock() - start) * factor:.3f} {unit}")

        return wrapper

    return decorator


def count_calls(
    *,
    logger: Callable[[str], None] = print,
    counter_factory: Callable[[], Callable[[], int]] = make_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Фабрика декоратора: рахує виклики через замикання-лічильник."""

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        """Кожна задекорована функція отримує власний лічильник."""
        counter = counter_factory()

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            """Збільшує лічильник і викликає функцію."""
            logger(f"[count_calls] {func.__name__} call #{counter()}")
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

Файл main.py

main.py рядки 1-41

```python
"""Точка входу лабораторної роботи №6, варіант 9."""

from closures import in_range, is_type, make_logger, non_empty, one_of, positive
from decorators import ValidationError, count_calls, timed, validate

log = make_logger("APP", print)


@timed("register", unit="us", logger=log)
@count_calls(logger=log)
@validate(
    name=[is_type(str), non_empty()],
    age=in_range(18, 65),
    salary=positive(),
    role=one_of("dev", "qa", "pm"),
)
def register_employee(name: str, age: int, salary: float, role: str = "dev") -> str:
    """Реєструє працівника і повертає опис запису."""
    return f"{name.strip().title()} ({role}), {age} р., {salary:.2f} грн"


def run() -> None:
    """Демонструє валідацію, таймінг і лічильник викликів."""
    print(register_employee("ivan petrenko", 30, 25000))
    print(register_employee("olena", 41, 31000, role="pm"))
    for args in [("", 17, -5.0, "ceo"), ("Petro", 70, 1000.0, "qa")]:
        try:
            register_employee(*args)
        except ValidationError as error:
            print(f"Помилка валідації: {error}")
    print(f"Метадані збережено: {register_employee.__name__}")
    print(f"Докстрінг: {register_employee.__doc__}")


def main() -> None:
    """Запуск демонстрації декораторів."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_closures.py

test_closures.py рядки 1-42

```python
"""Тести замикань і валідаторів."""

import pytest

from closures import (
    Validator,
    in_range,
    is_number,
    is_type,
    make_counter,
    make_logger,
    make_memory_sink,
    non_empty,
    one_of,
    positive,
)


def test_counter_keeps_state() -> None:
    """Тест: лічильник зберігає стан, різні лічильники незалежні."""
    a, b = make_counter(10), make_counter()
    assert [a(), a(), b()] == [11, 12, 1]


def test_logger_with_injected_sink() -> None:
    """Тест: логер пише в ін'єктований sink з префіксом."""
    write, history = make_memory_sink()
    log = make_logger("INFO", write)
    log("ready")
    log("done")
    assert history() == ("[INFO] ready", "[INFO] done")


def test_history_is_snapshot() -> None:
    """Тест: history повертає незмінну копію."""
    write, history = make_memory_sink()
    write("a")
    snapshot = history()
    write("b")
    assert snapshot == ("a",) and history() == ("a", "b")
```

test_closures.py рядки 43-67

```python
@pytest.mark.parametrize(
    ("validator", "value", "ok"),
    [
        (in_range(0, 10), 5, True),
        (in_range(0, 10), 0, True),
        (in_range(0, 10), 10.5, False),
        (in_range(0, 10), "5", False),
        (positive(), 0.1, True),
        (positive(), 0, False),
        (positive(), True, False),
        (non_empty(), "x", True),
        (non_empty(), "   ", False),
        (non_empty(), [], False),
        (non_empty(), 5, False),
        (is_type(int, float), 3.5, True),
        (is_type(int), "3", False),
        (one_of("a", "b"), "b", True),
        (one_of("a", "b"), "c", False),
        (is_number(), 7, True),
        (is_number(), False, False),
    ],
)
def test_validators(validator: Validator, value: object, ok: bool) -> None:
    """Тест: граничні та некоректні значення для кожного валідатора."""
    assert (validator(value) is None) is ok
```

Файл tests/test_decorators.py

test_decorators.py рядки 1-61

```python
"""Тести декораторів @validate, @timed, @count_calls та їх композиції."""

import inspect
from collections.abc import Callable

import pytest

from closures import in_range, is_type, make_counter, make_memory_sink, non_empty
from decorators import ValidationError, count_calls, timed, validate


def fake_clock(*ticks: float) -> Callable[[], float]:
    """Детермінований годинник, що повертає задані значення по черзі."""
    values = iter(ticks)
    return lambda: next(values)


def test_validate_accepts_valid_arguments() -> None:
    """Тест: коректні аргументи проходять."""

    @validate(x=in_range(0, 10))
    def double(x: int) -> int:
        """Подвоює число."""
        return x * 2

    assert double(4) == 8
    assert double(x=10) == 20


def test_validate_rejects_and_collects_errors() -> None:
    """Тест: усі помилки збираються в одне повідомлення."""

    @validate(name=[is_type(str), non_empty()], age=in_range(0, 120))
    def make(name: str, age: int) -> str:
        """Створює опис."""
        return f"{name}:{age}"

    with pytest.raises(ValidationError) as info:
        make("", 200)
    message = str(info.value)
    assert "name: не може бути порожнім" in message
    assert "age: має бути в межах" in message


def test_validate_checks_default_values() -> None:
    """Тест: значення за замовчуванням теж перевіряються."""

    @validate(limit=in_range(1, 5))
    def fetch(limit: int = 10) -> int:
        """Повертає ліміт."""
        return limit

    with pytest.raises(ValidationError):
        fetch()
    assert fetch(3) == 3


def test_validate_unknown_parameter() -> None:
    """Тест: правило для неіснуючого параметра - TypeError при декоруванні."""
    with pytest.raises(TypeError):
```

test_decorators.py рядки 62-120

```python
        @validate(missing=non_empty())
        def f(x: int) -> int:
            """Нічого не робить."""
            return x


def test_validate_does_not_call_function_on_error() -> None:
    """Тест: при помилці валідації функція не викликається."""
    calls = make_counter()

    @validate(x=in_range(0, 1))
    def f(x: int) -> int:
        """Рахує виклики."""
        return calls()

    with pytest.raises(ValidationError):
        f(5)
    assert f(1) == 1


def test_timed_uses_clock_and_logger() -> None:
    """Тест: таймінг без реального часу."""
    write, history = make_memory_sink()

    @timed("work", unit="ms", logger=write, clock=fake_clock(0.0, 0.012345))
    def work(x: int) -> int:
        """Квадрат числа."""
        return x * x

    assert work(5) == 25
    assert history() == ("[timed] work took 12.345 ms",)


def test_timed_logs_on_exception() -> None:
    """Тест: час логується навіть якщо функція кинула виняток."""
    write, history = make_memory_sink()

    @timed(unit="s", logger=write, clock=fake_clock(1.0, 3.5))
    def boom() -> None:
        """Завжди падає."""
        raise RuntimeError("boom")

    with pytest.raises(RuntimeError):
        boom()
    assert history() == ("[timed] boom took 2.500 s",)


def test_count_calls_state() -> None:
    """Тест: кожна функція має власний лічильник у замиканні."""
    write, history = make_memory_sink()

    @count_calls(logger=write)
    def a() -> None:
        """Функція a."""

    @count_calls(logger=write)
    def b() -> None:
        """Функція b."""
```

test_decorators.py рядки 121-182

```python
    a()
    a()
    b()
    assert history() == (
        "[count_calls] a call #1",
        "[count_calls] a call #2",
        "[count_calls] b call #1",
    )


def test_count_calls_injected_counter() -> None:
    """Тест: фабрика лічильника передається як залежність."""
    write, history = make_memory_sink()

    @count_calls(logger=write, counter_factory=lambda: make_counter(100))
    def f() -> None:
        """Функція f."""

    f()
    assert history() == ("[count_calls] f call #101",)


@pytest.mark.parametrize(
    "decorator",
    [
        validate(x=in_range(0, 10)),
        timed(logger=lambda _: None),
        count_calls(logger=lambda _: None),
    ],
)
def test_wraps_preserves_metadata(
    decorator: Callable[[Callable[[int], int]], Callable[[int], int]],
) -> None:
    """Тест: __name__, __doc__ і __wrapped__ збережено."""

    def original(x: int) -> int:
        """Оригінальний докстрінг."""
        return x

    wrapped = decorator(original)
    assert wrapped.__name__ == "original"
    assert wrapped.__doc__ == "Оригінальний докстрінг."
    assert inspect.unwrap(wrapped) is original


def test_decorator_order_matters() -> None:
    """Тест: порядок композиції змінює, чи рахуються невалідні виклики."""
    write_outer, outer_log = make_memory_sink()
    write_inner, inner_log = make_memory_sink()

    @count_calls(logger=write_outer)
    @validate(x=in_range(0, 1))
    def counted_first(x: int) -> int:
        """Лічильник зовні: рахує і невалідні виклики."""
        return x

    @validate(x=in_range(0, 1))
    @count_calls(logger=write_inner)
    def validated_first(x: int) -> int:
        """Валідація зовні: невалідні виклики не доходять до лічильника."""
        return x
```

test_decorators.py рядки 183-188

```
183     for f in (counted_first, validated_first):
184         with pytest.raises(ValidationError):
185             f(5)
186         f(1)
187     assert len(outer_log()) == 2
188     assert len(inner_log()) == 1
```

Запуск програми

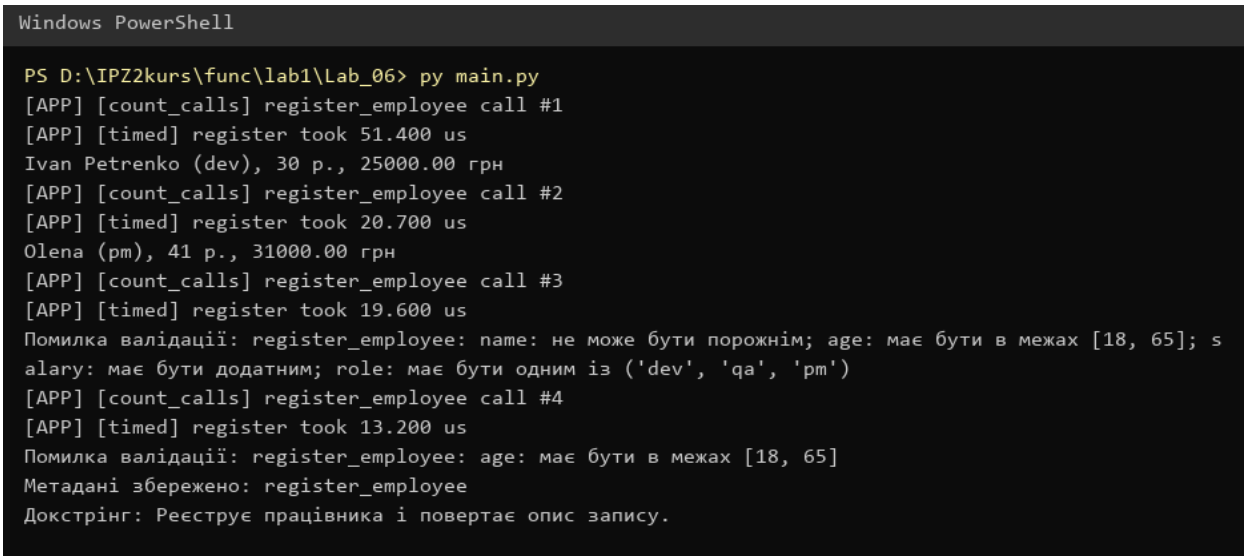


Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_06> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_06
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 33 items

tests/test_closures.py::test_counter_keeps_state PASSED                  [  3%]
tests/test_closures.py::test_logger_with_injected_sink PASSED            [  6%]
tests/test_closures.py::test_history_is_snapshot PASSED                  [  9%]
tests/test_closures.py::test_validators[check-5-True] PASSED             [ 12%]
tests/test_closures.py::test_validators[check-0-True] PASSED             [ 15%]
tests/test_closures.py::test_validators[check-10.5-False] PASSED         [ 18%]
tests/test_closures.py::test_validators[check-5-False0] PASSED           [ 21%]
tests/test_closures.py::test_validators[check-0.1-True] PASSED           [ 24%]
tests/test_closures.py::test_validators[check-0-False] PASSED            [ 27%]
tests/test_closures.py::test_validators[check-True-False] PASSED         [ 30%]
tests/test_closures.py::test_validators[check-x-True] PASSED             [ 33%]
tests/test_closures.py::test_validators[check-   -False] PASSED          [ 36%]
tests/test_closures.py::test_validators[check-value9-False] PASSED       [ 39%]
tests/test_closures.py::test_validators[check-5-False1] PASSED           [ 42%]
tests/test_closures.py::test_validators[<lambda>-3.5-True] PASSED        [ 45%]
tests/test_closures.py::test_validators[<lambda>-3-False] PASSED         [ 48%]
tests/test_closures.py::test_validators[<lambda>-b-True] PASSED          [ 51%]
tests/test_closures.py::test_validators[<lambda>-c-False] PASSED         [ 54%]
tests/test_closures.py::test_validators[<lambda>-7-True] PASSED          [ 57%]
tests/test_closures.py::test_validators[<lambda>-False-False] PASSED     [ 60%]
tests/test_decorators.py::test_validate_accepts_valid_arguments PASSED   [ 63%]
tests/test_decorators.py::test_validate_rejects_and_collects_errors PASSED [ 66%]
tests/test_decorators.py::test_validate_checks_default_values PASSED     [ 69%]
tests/test_decorators.py::test_validate_unknown_parameter PASSED         [ 72%]
tests/test_decorators.py::test_validate_does_not_call_function_on_error PASSED [ 75%]
tests/test_decorators.py::test_timed_uses_clock_and_logger PASSED        [ 78%]
tests/test_decorators.py::test_timed_logs_on_exception PASSED            [ 81%]
tests/test_decorators.py::test_count_calls_state PASSED                  [ 84%]
tests/test_decorators.py::test_count_calls_injected_counter PASSED       [ 87%]
tests/test_decorators.py::test_wraps_preserves_metadata[decorator0] PASSED [ 90%]
tests/test_decorators.py::test_wraps_preserves_metadata[decorator1] PASSED [ 93%]
tests/test_decorators.py::test_wraps_preserves_metadata[decorator2] PASSED [ 96%]
tests/test_decorators.py::test_decorator_order_matters PASSED            [100%]

============================== 33 passed in 0.08s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

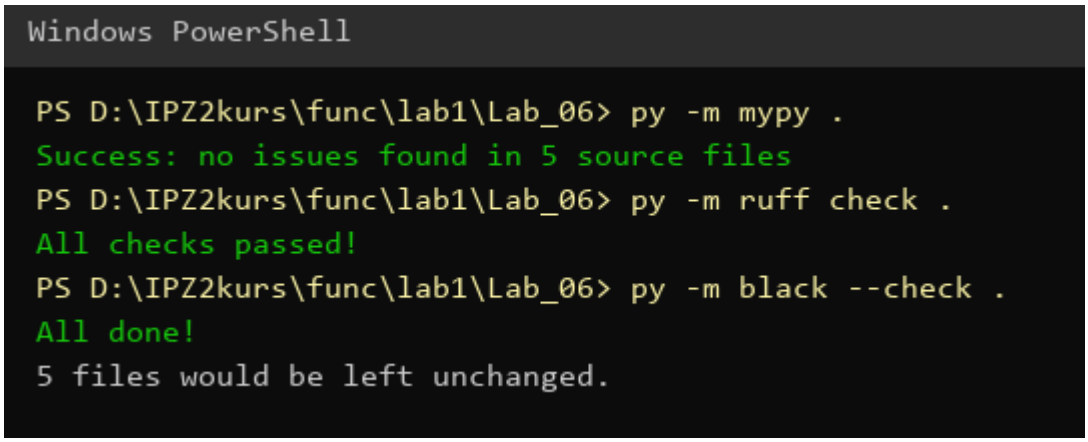


Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи реалізовано замикання зі збереженням стану та три параметризовані декоратори: валідації аргументів, таймінгу і підрахунку викликів. Валідатори створюються фабриками-замиканнями і передаються в декоратор як Callable, тому правила перевірки легко комбінувати. Ін'єкція годинника і логера зробила тести детермінованими, а functools.wraps і ParamSpec зберегли метадані та типи задекорованих функцій.